

Productbiografie - Sabrina

Opdracht

De opdracht is om voor Marjolijn, onze opdrachtgever, binnen haar project (Inter)Facing the Hybrid City een animatietool te ontwikkelen die experimenteren mogelijk maakt voor haar project. De tool moet aansluiten op haar onderzoek en de gebruiker in staat stellen om op een toegankelijke en flexibele manier typografische animaties te maken en te verkennen.

Mijn leerdoelen voor de meesterproef:

1. Tijdens het samenwerken mijn belangen beter benoemen en de andere daarin ook de ruimte in te geven. Zodat de communicatie in groepsverband beter voor iedereen prettig verloopt.
2. Aan het einde van de meesterproef wil ik duidelijkere code schrijven, zodat het overdraagbaar en makkelijk te begrijpen is voor een ander. Zeker met het schrijven van javascript code, ik begrijp nu wat ik zelf doe, maar andere moeten het ook nog snappen.
3. Daarnaast wil ik ook werken aan visuele esthetiek van een website, door mooie en interessante features eraan toe te voegen, die passend zijn bij het project. Een beetje het randje opzoeken van mijn kunnen en daarbij ook nog blijven werken aan mijn leerdoel van afgelopen weken het schrijven van redelijk complexe javascript/css code die ik ook begrijp.

Zie de reflectie hoe ik hier aan heb gewerkt 

Sommige codeblokken kunnen er in het huidige nieuwe project anders uitzien omdat ze gerefactord zijn; het originele heb ik hieronder voor mijn eigen proces bijgehouden.

Inhoudsopgave

Opdracht

Mijn leerdoelen voor de meesterproef.

Inhoudsopgave

Week 1

Afspraken maken + debrief

Afspraken binnen het groepje

Timeline functionaliteit

Recap week 1:

Week 2

Keyframe icon/tracks + draggable feature

Duration timeline indicator

Update range inputs + flyhead fx

Update range inputs

Recap week 2

Gesprek met opdrachtgever

Week 3

Duration veranderen

Code review met Cinda

Testen

Recap week 3

Gesprek met opdrachtgever

Week 4

Keyframe snap met buttons

Review button feature

Inklapbare tijdlijn

Outline fx functionality

Recap week 4

Gesprek met opdrachtgever

Week 5

Canvas selecteerbaar maken

Overdrachtsdocument

Recap week 5

Eindepresentatie met opdrachtgever

EXPO

Reflectie meesterproef

Meest trots

Had beter gekund

Reflectie Leerdoelen

Leerdoel 1

Leerdoel 2

Week 1

Afspraken maken + debrief

We zijn deze week gestart met afspraken maken en vragen opstellen voor de opdrachtgever, omdat we de opdracht best vaag vonden. Dus we hadden een lijst opgesteld met vragen en een presentatie voorbereid waarmee we voor de eerste keer naar de opdrachtgever Marjolijn gingen.

Afspraken binnen het groepje

- Contact houden wij via WhatsApp
- Takenlijst en vanuit daar een branche aanmaken om daarin te werken
- Wekelijks de dev branch updaten met de al gemaakte en functionaliteiten die af zijn.
- Pull requests: In ieder geval Sabrina meekijkt/overzicht houdt met die requests en samen met diegene van de geschreven code merges oplost.
- Code in Engels, documentatie Nederlands.

Rolverdeling

- Contactpersoon opdrachtgever: **Senna**
- Code beheerder: **Sabrina**
- Overzicht bewaarder: **Jeppe**
- Notulist: **Senna & Mila** (voor nu)
- Planner + Wiki beheerder **Kerr**
- Design beheerder: **Mila**

Timeline functionaliteit

Deze week heb ik een start gemaakt aan de tijdlijn, om deze visueel in beeld te krijgen voor de animation tool. Daarvoor heb ik de keuze gemaakt om GSAP te gebruiken; na wat onderzoek kwamen wij er allemaal achter dat dit in de

toekomst met de functionaliteiten ook handig kan werken en GSAP heeft ontzettend veel te bieden.

Daar heb ik de volgende code toen als opzet geschreven

```
// https://gsap.com/docs/v3/GSAP/Timeline/
// Define the timeline
const timeline = gsap.timeline({
  paused: true, //Making timeline that starts paused
  onUpdate: ()=>{ //When the animation automatically plays the slider updates on the progress of the animation duration
    if(timelineSlider) {
      timelineSlider.value = timeline.progress() * 100
    }
  },
});

// Define animation (this is a placeholder to try the slider functionality)
timeline.to(".text",{
  x: 400, //Translate
  rotation: 360, //Rotate
  duration: 3, //Duration
  ease: "none", //No ease
});

// Connect the interface to the engine
const timelineSlider = document.querySelector("#timeline-slider");

// Get play button
const playButtonTimeline = document.querySelector('.play-animation');

// create the update when input changes
timelineSlider.addEventListener("input", ()=> {
  const progressTimeline = timelineSlider.value / 100;
```

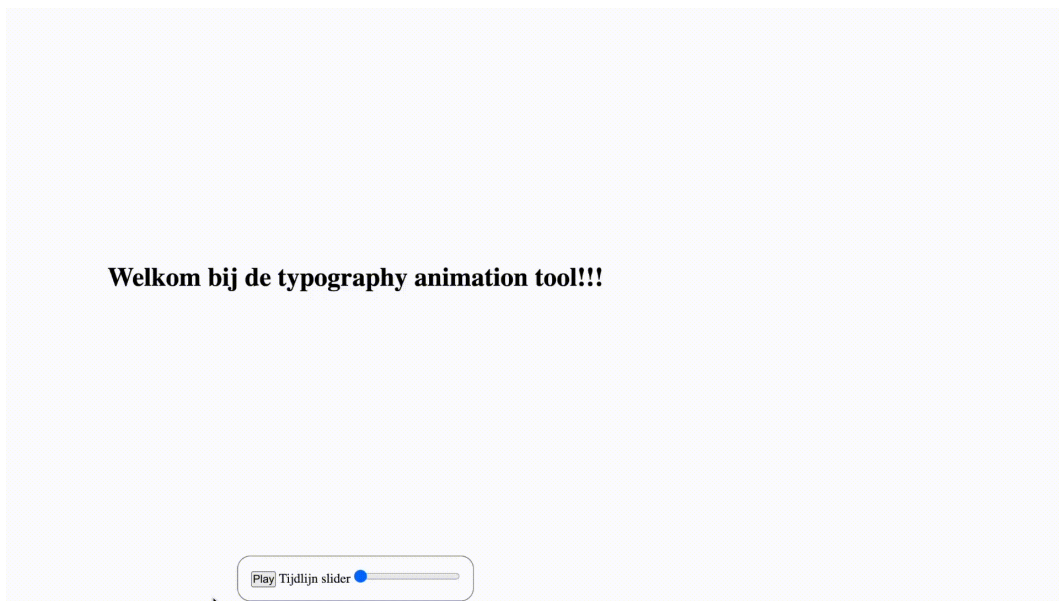
```

        timeline.progress(progressTimeline);
    });

    // To play/pause the whole animation at once
    let isPlaying = false; //Standard is false

    playButtonTimeline.addEventListener("click", ()=> {
        if(isPlaying === false) {
            timeline.play();
            playButtonTimeline.textContent = 'Pause';
            isPlaying = true;
        } else {
            timeline.pause();
            playButtonTimeline.textContent = 'Play';
            isPlaying = false;
        }
    })

```



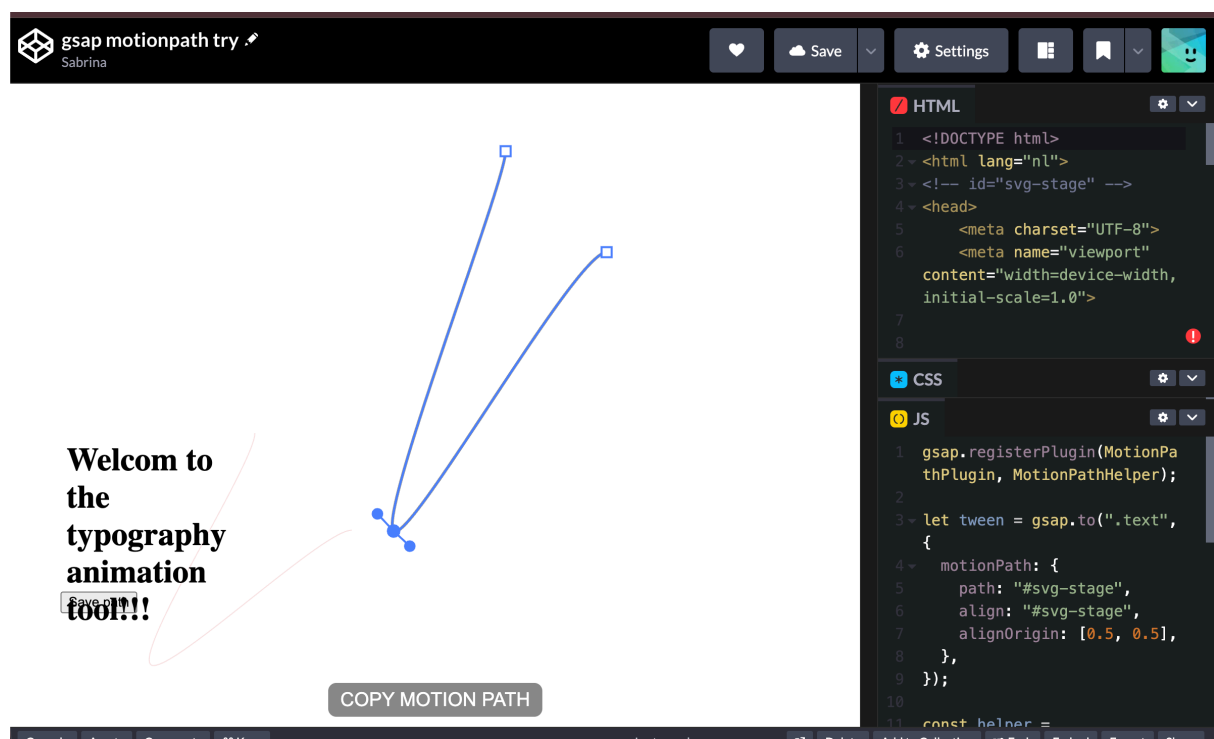
Daarnaast had ik omdat een van mijn leerdoelen is om goed overdraagbare code te schrijven ook even met Kerr meegekeken in hoe hij zoiets aanpakt. Hij gaf bijvoorbeeld als je met een if/else statement werkt en je wilt daarin iets met true or false terugkrijgen dan kan het altijd wel korter genoteerd worden.

Voor het overzicht een herbruikbaarheid de variabelen en aanroepen van functions die direct moeten doen ook allemaal bovenin zetten.

Ook ben ik deze week wat wezen experimenteren met MotionPath van GSAP omdat we nog in een zoekende fase waren met welke elementen we erin willen ben ik hier wat mee gaan proberen.

<https://codepen.io/Sabrina-z/pen/NPbgXeQ>

Hier zijn we uiteindelijk niet mee verder gegaan, later uit de tests bleek namelijk dat de opdrachtgever wil gaan voor low hanging fruit, en niet al te moeilijk. Mila had ook een coole grafische ease function gemaakt, daar gaf ze ook al bij aan dat ze liever gewoon een dropdown heeft.



Ook dit had een aantal haken en ogen, want hij maakt via deze een path, maar het update de svg nog niet waarin het moet komen en dat had wel mooi geweest als dat had gelukt nog.

Recap week 1:

Ik heb deze week veel dingen geleerd en onderzocht en geëxperimenteerd, waaronder GSAP en veel meer met javascript gedaan dan ik vooraf had verwacht en daar ben ik nu al trots op.

Week 2

In deze week ben ik bezig geweest met een ander leerdoel en dat is visueel esthetische elementen toevoegen die ik aan de hand van een functionaliteit maak. Daarnaast heb ik andere dingen netter gemaakt zoals dat de playhead meeschaalt en nog meer.

Keyframe icon/tracks + draggable feature

Daarvoor heb ik de kleine icons gemaakt die tevoorschijn komen op het moment dat je een input aanpast.



Dit heb ik gedaan door een ruitje te maken in css, dat heb ik gedaan door een **corner-shape** te gebruiken, maar achteraf bleek een polygon beter te zijn omdat je dat op het element zelf gebruikt en niet de border, wat ik met dat andere wel had gedaan.

```
.keyframe {  
  position: absolute;  
  
  clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);  
  background-color: var(--color-details);  
  
  --keyframe-size: 1em;  
  width: var(--keyframe-size);  
  left: calc(var(--p) * 100% - var(--keyframe-size) / 2);  
  aspect-ratio: 1;  
  align-self: center;  
}
```

Dus dat is het uiteindelijk geworden met de css. Maar voor de code in javascript om het te laten komen heb ik het volgende geschreven.

Uiteindelijk om de tracks te maken en de keypoints een plek te geven heb ik een function geschreven als visualizer, omdat je de tijdlijn ermee maakt en opbouwt op basis van de verschillende properties.

```
buildVisualizer() {  
  const visualizerContainer = document.querySelector(".time  
line-container"); // jouw DOM-element  
  visualizerContainer.innerHTML = " "  
  
  // Get the data from the json  
  this.timelineData.animations.forEach((animationData) => {  
    animationData.properties.forEach((propertyData) => {  
  
      // Create elements to create for each functionality a  
row  
      const row = document.createElement("div");  
      row.classList.add("row");  
  
      const track = document.createElement("div");  
      track.classList.add("track");  
  
      const trackText = document.createElement("p");  
      trackText.classList.add("track-label");  
  
      row.append(trackText, track)  
  
      trackText.innerHTML = `${propertyData.property}  
</span>`;  
  
      // Create for each keyframe point a point on the row  
      propertyData.keyframes.forEach((keyframe) => {  
        const point = document.createElement("div");  
        point.classList.add("keyframe");  
        point.style.left = `calc(${keyframe.progress * 10  
0}% - 7px)`;  
        track.appendChild(point);  
      });  
    });  
  });  
}
```

```

        // Add the elements to the html
        visualizerContainer.appendChild(row);
    });
};

}

```

Ook is het handig om deze keyframes naar een andere plek te kunnen verplaatsen en daar is eigenlijk GSAP ook wel heel fijn voor want zij hebben een draggable mogelijkheid wat hier goed van pas komt.

```

Draggable.create(point, {
    // Type of way you can dragg the element
    type: 'x',
    bounds: track,
    onClick() {
        // Use offsetwidth to get the length in pixels o
f the track
        const trackWidth = track.offsetWidth;
        // To get the new position "what point was start
ing point" + "the point i dragged it to"
        const pointX = this.x + (trackWidth * keyframe.p
rogress);

        // So it staves inside the track. math min so it
doesn't get above the 1 and below 0
        keyframe.progress = Math.max(0, Math.min(1, poin
tX / trackWidth));
    },
    onDragEnd(){
        const trackWidth = track.offsetWidth;
        const pointX = this.x + (trackWidth * keyframe.p
rogress)

        //https://developer.mozilla.org/en-US/docs/Web/J
avaScript/Reference/Global_Objects/Math/min
        //https://developer.mozilla.org/en-US/docs/Web/J
avaScript/Reference/Global_Objects/Math/max
    }
});

```

```

        keyframe.progress = Math.max(0, Math.min(1, pointX / trackWidth));

        // Rebuild the animation to the new keypoints
        this.buildAnimation();
    }
});
});

```

Duration timeline indicator

In eerste instantie voor de indicator had ik gebruik gemaakt van een span en het ophalen van de progress. En dit werkte eigenlijk wel goed zo, voor nu alleen later willen we dit nog gaan aanpassen, naar een aanpasbare duration. Alleen de vraag is nog of we dat in de toolpanel doen of toch bij de tijdlijn.

```

const progress = timeline.progress();
// Use a number to show into a string .toFixed(2) the 2 is
// for the decimals
// https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toFixed
const current = (progress * timeline.duration()).toFixed(2)
// Get the timelines current number / get the duration of
// the timeline
document.querySelector('.tl-time-start').textContent = `${current}s`
document.querySelector('.tl-time-end').textContent = `${timeline.duration().toFixed(2)}s`

```

Update range inputs + Playhead fix

Ik kwam er tijdens het testen van de tool zelf erachter dat de tijdlijn zelf het beste over de tracks heen kan liggen. Zo kan je namelijk een stuk makkelijker zien bij welke keyframes de animatie op dit moment is. Dit heb ik uiteindelijk opgelost door een observer toe te voegen die bijhoudt of de container langer wordt of niet. Zodra deze langer wordt moet de playhead/slider ook langer

worden. Dit werkte goed, en ik vond het ook wel leuk om deze webAPI een keer te gebruiken.

```
// Playhead element on the timeline increases it/s length w
hen the a track is added
function updatePlayheadHeight() {
// Get the height from the container that needs to be track
t for the height
// https://developer.mozilla.org/en-US/docs/Web/API/Elemen
t/getBoundingClientRect
const containerBottom = tracksContainer.getBoundingClientRect().bottom;
const sliderTop = timelineSlider.getBoundingClientRect().top;

// Devide by eachother so it doesn't get the wholee heigh
t
const height = containerBottom - sliderTop;
timelineSlider.style.setProperty('--height-playhead', `
${height}px`);
}

// API that tracks if an element changes size
// https://developer.mozilla.org/en-US/docs/Web/API/ResizeO
bserver
const observer = new ResizeObserver(updatePlayheadHeigh
t);
observer.observe(tracksContainer);

updatePlayheadHeight();
```

Update range inputs

Ook liepen we tegen een probleem aan dat de bars waarin je je animatie kan invullen gewoon bleven staan, dus daar heb ik ook een function voor geschreven. In eerste instantie snapte die dan van punt a naar punt b en daar was ik ook niet tevreden over dus kwam ik erachter dat je die waardes ertussen

kan ophalen met Gsap. En dat ziet er dan al meteen een stuk gebruiksvriendelijker uit.

```
function updateRangeInputs () {
  const animations = animationBuilder.timelineData.animations;
  const currentProgress = animationBuilder.getProgress();

  // Search for all the controls
  animationControls.forEach((control) => {
    const propertyName = control.dataset.property

    //First search for the animation in JSON
    const animation = animations.find(
      (animation) => animation.target === "el-1" //TODO ook
      voor andere stukken tekst die je dan selecteerd aanpassen.
    );

    if (!animation) {
      // Let the rangevalue be 0
      control.value = 0;
      return;
    }

    //Second look for the property in JSON
    const property = animation.properties.find(
      (property) => property.property === propertyName
    );

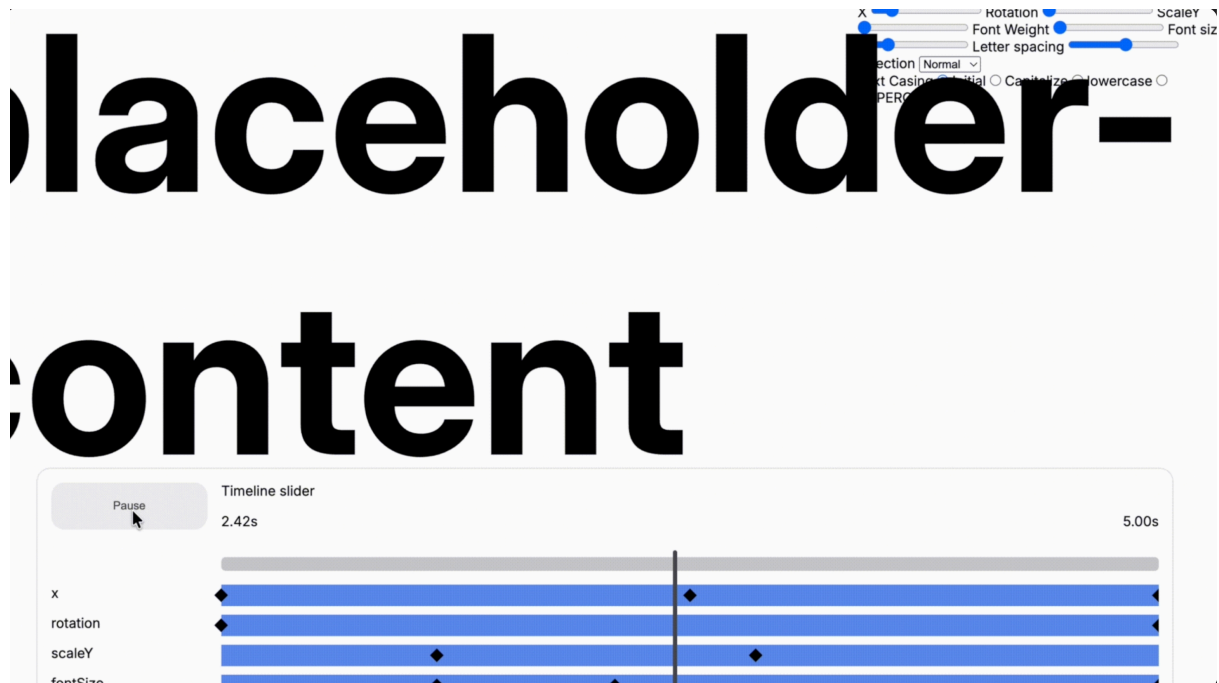
    if (!property) {
      control.value = 0;
      return;
    }

    const target = document.querySelector(".el-1");
    const currentValue = gsap.getProperty(target, propertyName)
  })
}
```

```

    control.value = currentValue
  })
}

```



Recap week 2

Deze week heb ik leren werken met mathmin en mathmax, het ophalen en bekijken van de json. Het gebruik maken van de webapi ResizeObserver. Dit kan in sommige gevallen echt heel handig zijn. Ook heb ik geleerd dat .toFixed() zich afrondt op... als je er een twee in zet dan zijn het twee decimalen. getBoundingClientRect() dat haalt de hoogte op van het element. En genoeg andere kleine dingen die nieuw voor mij waren. Er gaat een hele wereld zo voor mij open, omdat ik niet wist dat dit allemaal mogelijk was.

Gesprek met opdrachtgever

Deze week waren we op gesprek geweest met de opdrachtgever en hebben we laten zien wat wij hebben gemaakt. Daarin heb ik de tijdlijn en keyframes uitgelegd. En hebben we zo de basis functionaliteiten laten zien.

Ook zijn we toen in gesprek gegaan over de diverse variabelen binnen het project en was zij op dit moment niet tevreden met de rotation.

Week 3

Duration veranderen

We zijn uiteindelijk gegaan voor de duration onderin bij de tijdlijn, omdat je dit in afterEffects daar ook terug ziet, en het leek ons beter twee vliegen in een klap en alles wat in de toolbar staat toe te wijden aan alles wat je kan doen bij een element en alles wat te maken heeft met de tijdlijn daarbij te zetten.

```
setDuration(newTime){  
  this.#durationSeconds = newTime;  
  this.#timelineData.duration = newTime;  
  this.buildAnimation()  
}
```

Uiteindelijk hadden we een probleem tijdens het mergen dat de code er helemaal niet meer bijstond, dus ik heb ik het aan mijn medestudent gevraagd, want de code was op dit moment best veel veranderd, hoe ik dit dan moest doen.

```
document.querySelector('.tl-time-start').value = current;  
  
// Input starting point when the value is changed divide with the duration and set the new progress to the playhead  
document.querySelector('.tl-time-start').addEventListener  
( 'change', (e) => {  
  const time = parseFloat(e.target.value);  
  const progress = time / animationBuilder.timelineData.duration;  
  animationBuilder.setProgress(progress);  
});  
  
setDuration(newTime){  
  this.#durationSeconds = newTime;  
  this.#timelineData.duration = newTime;  
  this.buildAnimation()  
}
```

```
this.#timeline.duration(this.#durationSeconds);
```

Code review met elkaar

Deze week waren we ook op het punt gekomen waarin er zoveel veranderingen hadden plaatsgevonden, dat meerendeel van het groepje niet snapte wat er aan de hand was en wat er nu daadwerkelijk in de code gebeurd. Er stonden ook bijna geen comments bij en dat maakte het ook erg lastig om het te begrijpen.

Dus wij hebben deze week met het hele groepje 2 dagen uitgestrekt om hiervoor te gaan zitten, waarin Kerr de code aan ons heeft uitgelegd en waarin wij vragen konden stellen. Dit heeft wel geholpen, want ik begrijp het nu een stuk beter.

Tijdens deze gesprekken heb ik ook aan mijn leerdoel kunnen werken, namelijk dat ik voor mijn eigen belangen ben opgekomen om het voor mij ook duidelijk te maken. Daarbij gaf ik aan dat de comments echt heel erg helpen om het te begrijpen. En dat bepaalde namen anders benoemd konden worden (daar kom ik nu op terug, omdat ik bezig ben geweest met de code en het nu wel logisch vind)

Testen

Tegen het einde van de week hebben ik en Senna de tool getest met drie klasgenoten. We hebben getest op met het woord "system" waarin ze de opdracht kregen om bij de betekenis van dit woord een animatie te maken. Hierin konden wij best goed zien waar zij tegenaanliepen binnen de tool.

Zoals:

- Bepaalde functies werkte totaal niet met animeren
- Ze kwamen er niet achter dat de woorden gesleept konden worden
- Ook wisten ze niet dat de timeline verder naar beneden gescrollt konden worden.
- Sommige hadden de redo/undo button niet gebruikt

Dit waren wel hele handige inzichten en zeker om nog aan te passen voordat we het gesprek hebben met de opdrachtgever.

Recap week 3

Deze week zijn wij voornamelijk bezig geweest met het snappen en begrijpen van de algehele structuur van de code. Dat nam best wel wat tijd in beslag, maar hierdoor heb ik ook wederom weer veel geleerd. Over classes en over `map()`. De `map` lijkt eigenlijk veel op een array maar bij een `map` kan je gericht zoeken als het nodig is.

Gesprek met opdrachtgever

Ze was deze week erg enthousiast over onze tool en ze kon niet wachten om er zelf mee te experimenteren. We hadden tijdens dit gesprek ook besproken met het andere groepje dat dit wel degelijk twee nieuwe projecten zijn, omdat het te lastig wordt voor ons om dat samen te voegen zeker in de tijd die wij nu nog hebben.

Daarnaast hebben we Marjolijn onze opdrachtgever zelf even laten proberen en ze gaf ook aan dat bepaalde dingen nog miste en dat we andere dingen weer weg mochten laten. Dus deze inzichten nemen wij allemaal mee al werkende naar het einde toe. Daarnaast wederom weer bij het testen werkten een aantal functionaliteiten niet optimaal, dus die bugs moeten we ook nog zien aan te passen.

Week 4

Keyframe snap met buttons

Voor in de timeline hadden we besloten dat het wel handig zou zijn dat er buttons binnen komen waarin de gebruiker op kan klikken om naar een volgende keyframe te snappen.

Dit heb ik op mij genomen ik heb er een functie van gemaakt in eerste instantie kwam ik er niet helemaal uit, ik wilde het proberen met de `findClosest` maar dat werkte niet helemaal, daar had ik de volgende code voor geschreven.

```
function snapPlayheadToKeyframe (propertyName) {  
  // With the function you got the ID of the element  
  const elementId = getFirstElementId();  
  if (!elementId) return;  
  
  const keyframes = animation.getKeyframes(elementId, propertyName);  
  const currentProgress = parseFloat(timelineSlider.value)
```

```

/ 100;
const snapDistance = 2;

// https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/abs
// Math abs finds the difference in numbers between two points
const findClosest = keyframes.find((keyframe) => Math.abs(
keyframe.progress - currentProgress) <= snapDistance);

if(findClosest) {
  timelineSlider.value = findClosest.progress * 100;
} else {
  console.log("het werkt niet")
}
}

```

Toen heb ik uiteindelijk wat anders geprobeerd, omdat je binnen de map van de Json moet gaan lopen om te kijken welke keyframe er een waarde heeft die na het huidige punt komt van de playhead. Dus hieronder zie je een voorbeeld van de vorige keyframe, de volgende is precies hetzelfde maar dan met de waardes andersom. Uiteindelijk bleef de playhead hangen op een bepaald punt dit hebben we gefixt met een 0.00001 ervanaf te halen.

```

animationData.getElements().keys().forEach(element =>{
  const animationMap = animationData.getAnimations(element);
  if(!animationMap) return;

  animationMap.values().forEach(keyframes => {
    keyframes.forEach( keyframe => {
      if (keyframe.progress < currentProgress - 0.0001)
    {
      if (prevProgress === null || keyframe.progress > prevProgress) {
        prevProgress = keyframe.progress;
      }
    }
  }
}

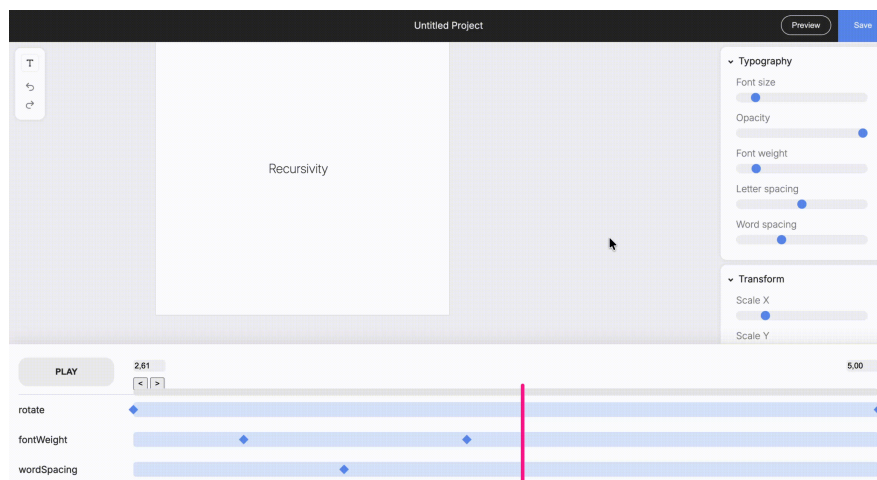
```

```

    })
  })
})

if(prevProgress !== null){
  player.setProgress(prevProgress)
} else {
  player.setProgress(0)
}

```



Preview button feature

In week vier ben ik bezig geweest aan een preview, omdat wij tijdens het gesprek met de opdrachtgever hadden besloten om het echt te zien als twee opdrachten. Tonen wij nu alleen de animatie zonder alle andere elementen erin, waardoor je echt je focus kan leggen op de animatie in plaats van de tijdlijn en de toolbar. We waren in eerste instantie van plan om hier dan een html opzet in te zetten van het andere groepje, maar dat doen we na het vorige gesprek uiteindelijk toch niet.

Ik was hieronder begonnen met een popover element maken, hier komen namelijk extra functionaliteiten bij kijken voor een eventuele screenreader gebruiker. Alleen dit werkte nog niet helemaal, hij toont wel een frame van de animatie, maar hij speelt nog niet af en die `.play()` werkte ook niet.

```

import { player, AnimationData } from "../timeline.js";
import AnimationPlayer from "../animations/animation-player.js";

// -----
// MARK: PREVIEW CANVAS
// -----

const popover = document.querySelector('#popover-original');

document.getElementById('open-button').addEventListener('click', () => {
    const originalCanvas = document.querySelector('#original-canvas');
    // https://developer.mozilla.org/en-US/docs/Web/API/Node/cloneNode
    // Makes a clone from the html attribute
    const popoverCanvas = originalCanvas.cloneNode(true);

    popoverCanvas.removeAttribute('id');

    // Put the clone inside the popover attribute.
    const popoverContent = document.querySelector('#popover-content');
    popoverContent.innerHTML = '';
    popoverContent.appendChild(popoverCanvas)

    // Maak een nieuwe player aan op de geclonede canvas met dezelfde animatiedata
    const previewCanvas = popoverCanvas.querySelector('.content-canvas');
    const previewPlayer = new AnimationPlayer(previewCanvas, animationData);

    popover.showPopover();

```

```
previewPlayer.play();  
});
```

Inklapbare tijdlijn

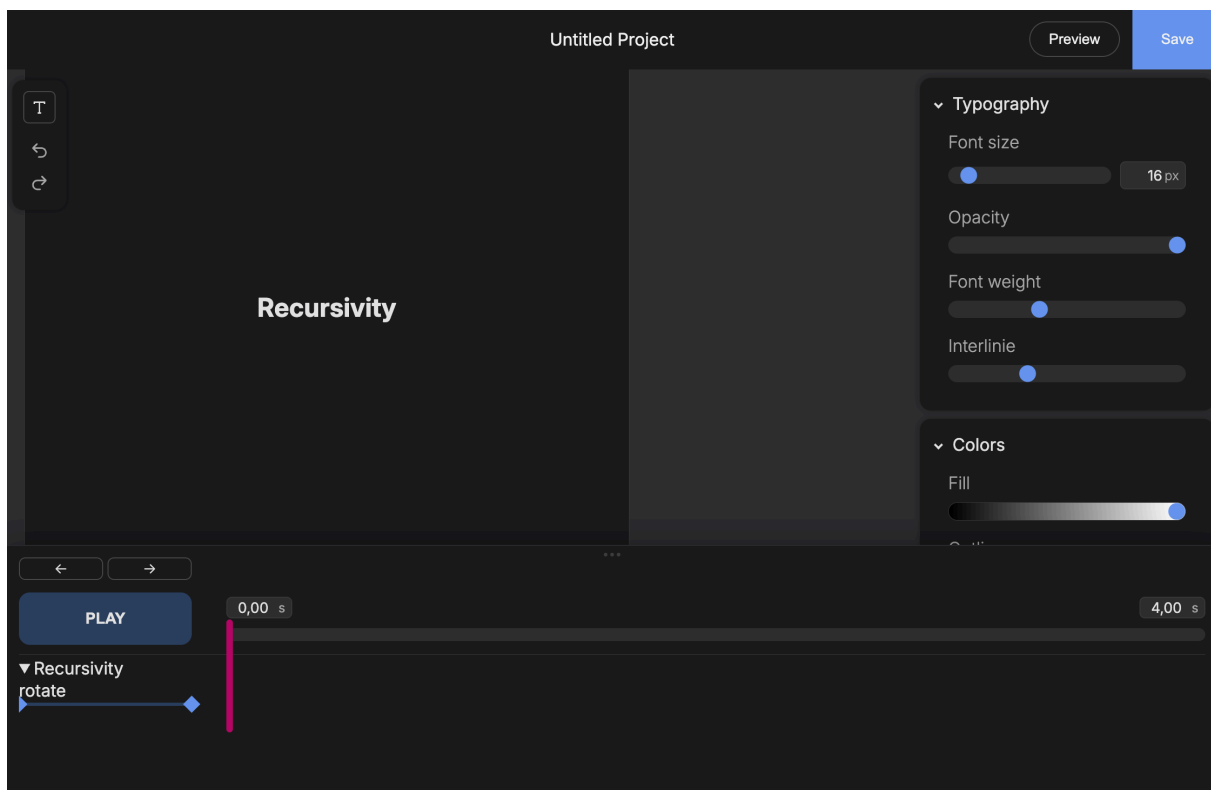
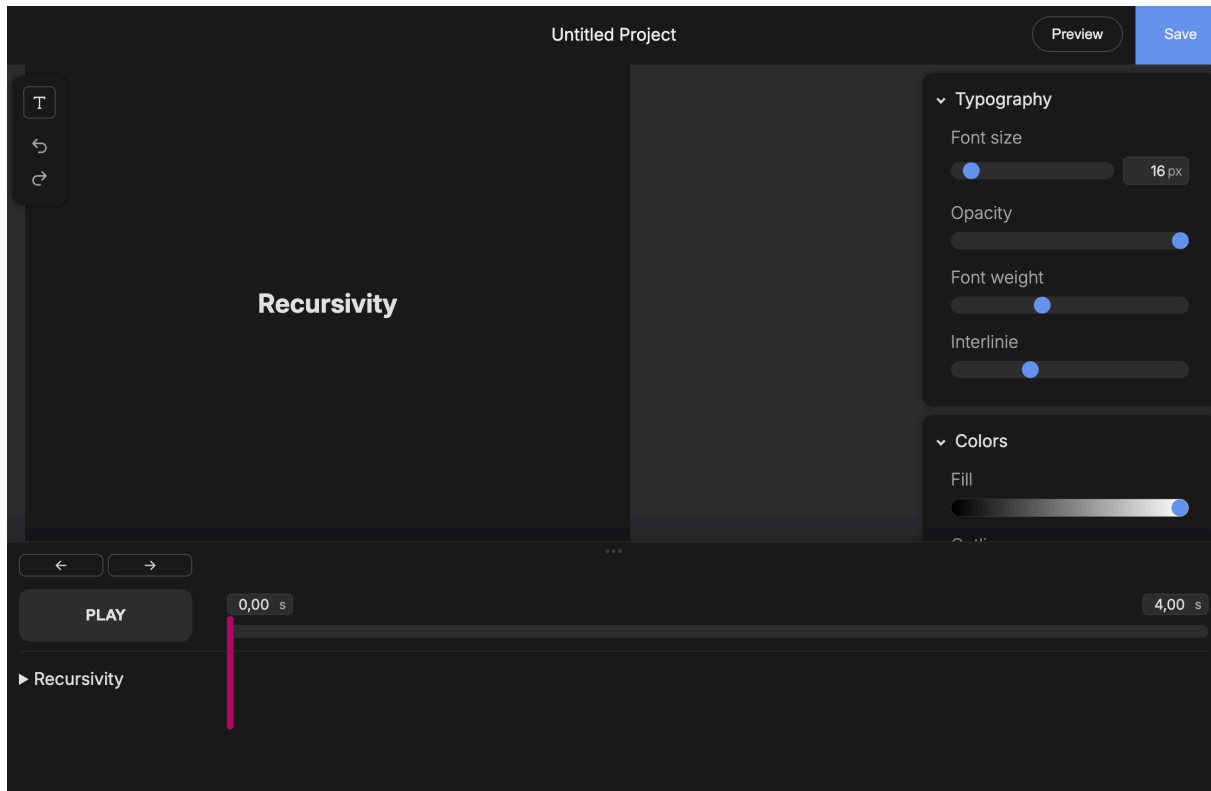
Als laatste deze week heb ik mij bezig gehouden met een functionaliteit waarbij de tijdlijn in kan klappen als er te veel regels zijn. Zo creëer je per woord een duidelijkheid over welke tracks erbij horen.

```
// Size is the length of the map  
if (animationData.getAnimation().animations.size === 0) return;  
  
// Loopen through each element to create a track for every  
single element and property  
animationData.getAnimation().elements.forEach((value, elementId) =>{  
  
    // Within the timeline create a dialog and put it in the  
    container  
    const timelineSection = document.createElement('details');  
    timelineSection.classList.add("timeline-section")  
  
    // Add a summary which is the elementId or value and set  
    it in the details  
    const timelineSectionSummary = document.createElement('summary');  
    timelineSectionSummary.innerHTML = value;  
    timelineSection.appendChild(timelineSectionSummary);  
  
    container.appendChild(timelineSection);  
});
```

Dit heb ik toegevoegd aan de build visualizer, alleen in eerste instantie had ik dit staan voor de summary, alleen dat gaf de id weer, als ik dit aanpaste naar value dan werkte dit niet meer. Uiteindelijk heeft Kerr met mij meegekeken om die aanpassing te kunnen maken.

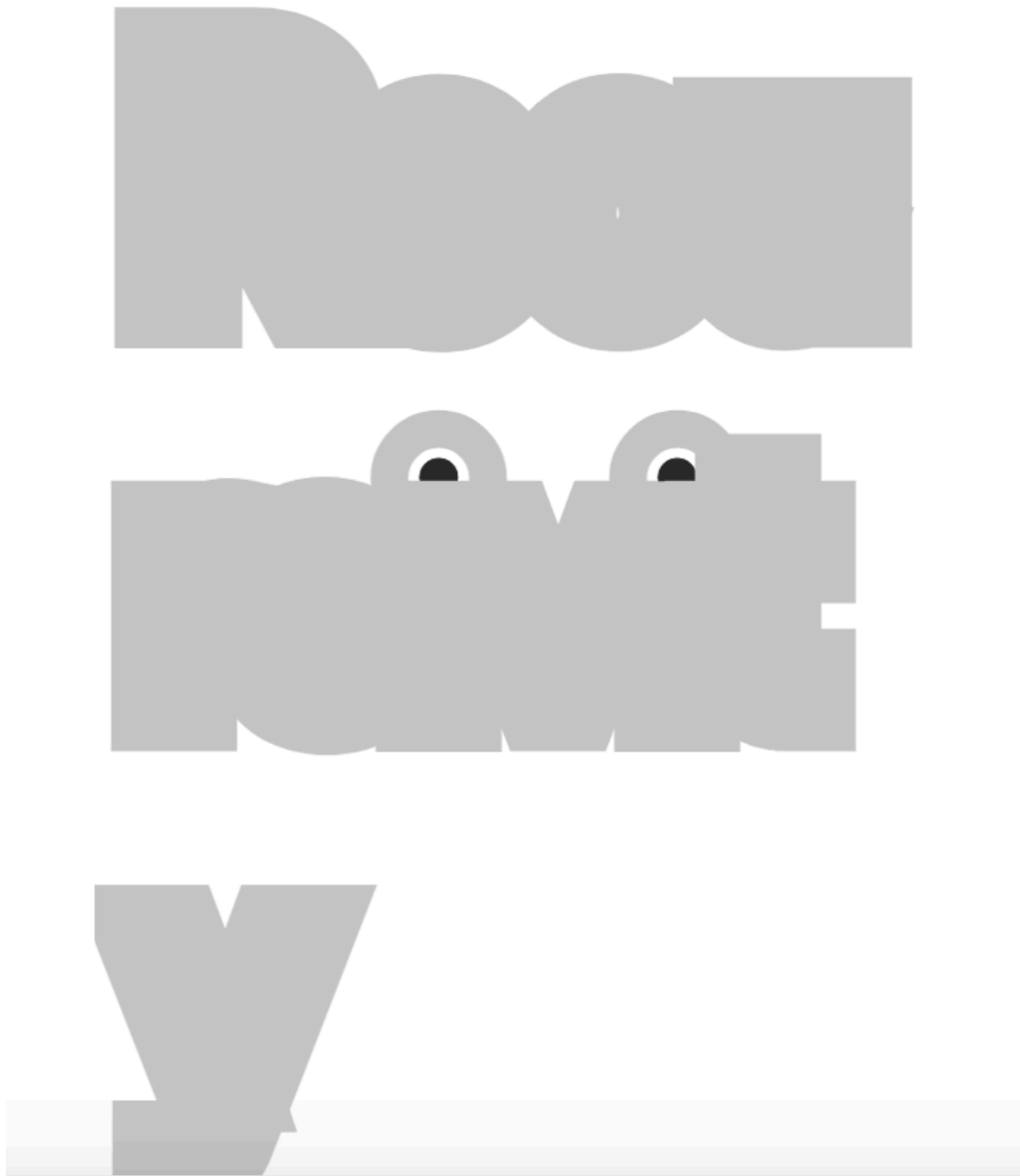
```
timelineSectionSummary.innerHTML = `${elementId}`;
```

Zie onderin de foto bij de tijdlijn recursivity [!\[\]\(6ae6c59c929f443b2be20fc4e10f1c3a_img.jpg\)](#)

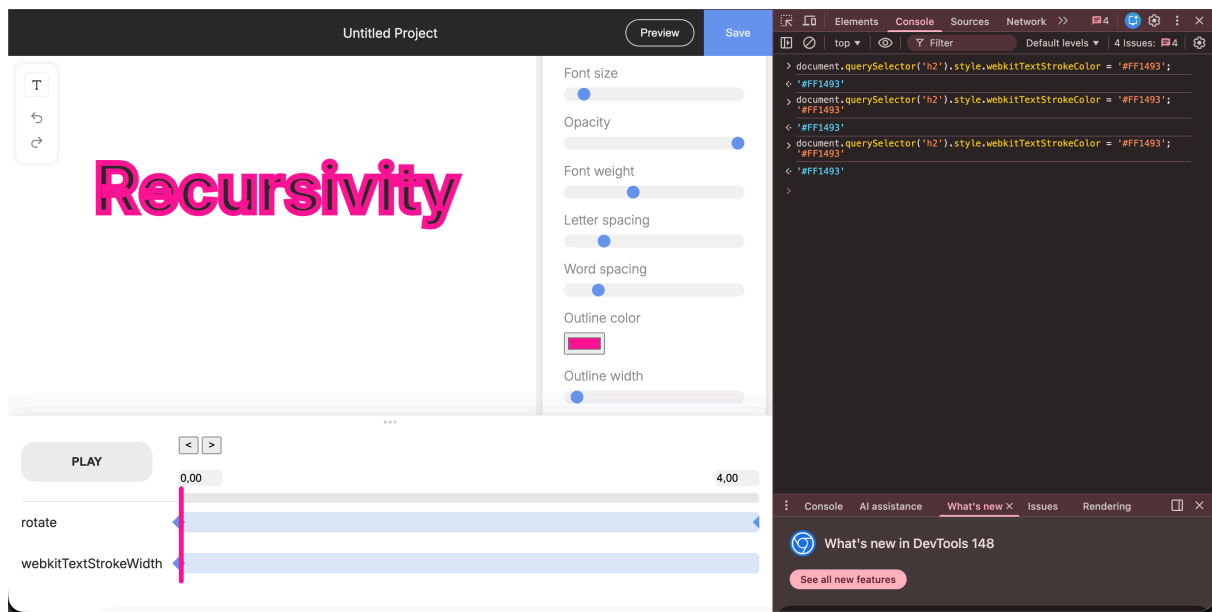


Outline fix functionality

Ook heb ik de outline voor de woorden gefixt. In eerste instantie werd deze heel breed en groot. Waardoor het net blokken leken.



Dus uiteindelijk heb ik de steps aangepast van de width in de html en kan je max een kleine outline maken. Hieronder een voorbeeld daarvan, alleen de roze kleur zit er nu niet meer in, omdat een van de vereiste zwart/wit was. Dus we hebben alles wat tussen zwart en wit zit als optie gegeven.



Recap week 4

Deze week heb ik mij voornamelijk gefocust op het toepassen en toevoegen van functionaliteiten van de huidige structuur. Dit nam soms even wat meer tijd in beslag om te kijken welke elementen ik daadwerkelijk nu het beste kon gebruiken. Maar ik kwam er hierdoor wel achter dat je binnen die class functies kan maken en die ook op andere plekken hergebruiken, en dat maakt het al een stuk makkelijker om een nieuwe functionaliteit toe te voegen als je al een functie hebt die de elementen bijvoorbeeld ophaalt die geanimeerd kunnen worden.

Gesprek met opdrachtgever

De opdrachtgever was wederom weer blij met wat wij hadden gemaakt, ze was heel enthousiast over de diversiteit in mogelijkheden, maar gaf wel echt duidelijk aan dat dit er genoeg zijn.

Ook hebben we voor de duidelijkheid nogmaals even besproken hoe wij dit gaan opleveren en dat is de link en een ZIP-bestandje met daarin een uitleg hoe je te werk moet gaan met dit project mocht je de code willen aanpassen.

Week 5

Deze week heb ik een aantal fixes gedaan binnen de code en heb ik mijn focus gericht op het document voor de overdracht aan Marjolijn en alvast een start gemaakt aan de design rationale.

Canvas selecteerbaar maken

Zoals het selecteerbaar maken van het canvas en deze ook een border geven als deze geselecteerd is. In eerste instantie had ik het zonder die if else gedaan, maar nu onthoudt hij zagezegd de state in een true of false waardoor het toevoegen van een styling makkelijker gaat.

```
canvasContainer.addEventListener('dblclick', (event) => {
  if (selectedCanvas === false) {
    if (event.target !== canvasContainer) return; // If double clicked on selected text, don't select canvas

    let canvasId = null

    animationData.getElements().forEach((el, id) => {
      if (event.target !== canvasContainer) return;
      // Search for the type canvas within the elements
      if (el.type === "canvas") {
        canvasId = id
      }

      // If it doesn't exist then return null
      if (!canvasId) return;

      // When it's clicked then the activeElement becomes the canvas id to animate the background
      activeElementId = canvasId

      canvas.style.outline = "3px solid #6495ED";
      // Gives the selected id from the canvas
      animationData.setSelectedText(canvas, canvasId)
    });

    selectedCanvas = true;
  } else {
    canvas.style.outline = "none";
    selectedCanvas = false;
  }
});
```

```
}  
})
```

Op een gegeven moment werkte dit ook niet meer met mergen en i.v.m. de json. Waardoor ik aan de hand van console.log(s) heb gekeken waar het probleem zat. En dat was dat hij het hele element type niet kon vinden in het JSON-bestand.

Dit heb ik uiteindelijk samen met Kerr even opgelost, want ik kwam er niet uit met het fixen van die bug. Uiteindelijk moesten we checken of er in een array het element.type canvas zit en zo niet dan moet er eentje aangemaakt worden.

```
// Checks in the array of there is an element with type canvas  
const hasCanvasElement = elements.values().find((element)  
=> element.type === "canvas");  
  
// If not then create an element canvas with type canvas  
if (!hasCanvasElement) {  
  this.#animation.createElement("canvas", "canvas");  
}
```

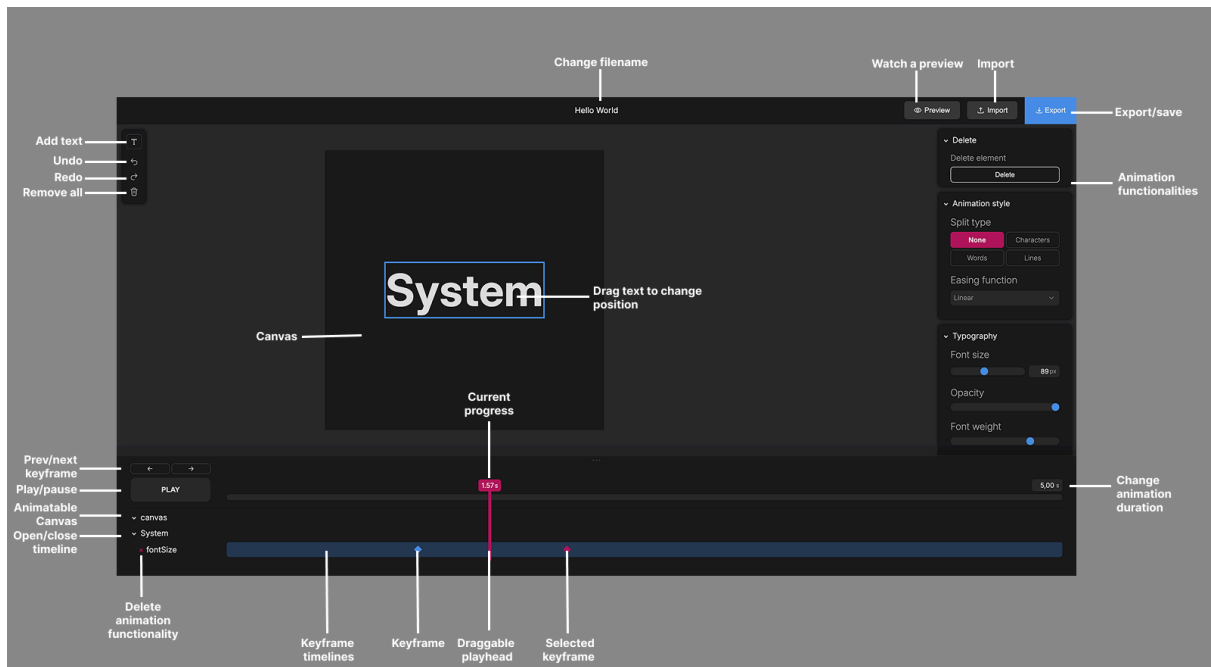
Eigenlijk best logisch.

Overdrachtsdocument

Voor een opzetje heb ik hierbij wel ChatGPT gebruikt om te vragen wat er in een overdrachts-readme moet komen staan. Zo heb ik wat handvatten om dit te gaan maken.

Zoals een kopje met functionaliteiten, waarin duidelijk is wat welke button doet. Het leek mij onduidelijk om dit in woorden uit te leggen dus dit heb ik visueel gemaakt in Figma.

Uiteindelijk heeft Jeppe er nog even naar gekeken en wat opmerkingen en punten bijgevoegd die ook handig zijn en die heb ik vervolgens aangevuld. Zoals een functionaliteiten kopje, waarin duidelijk is wat welke button doet. Het leek mij onduidelijk om dit in woorden uit te leggen dus dit heb ik visueel gemaakt in Figma.



Daarna hebben we samen met het groepje nog gekeken naar de readme en hebben we wat punten besproken die aangevuld moesten worden of juist toegevoegd. Dit was wel heel fijn, op een gegeven moment zie je het zelf niet helemaal meer en dan is het fijn dat er andere met je meekijken.

Recap week 5

Deze week hebben we als groep zijnde allemaal gewerkt aan diverse bugs oplossen, zodat het uiteindelijk allemaal goed werkte en zijn wij aan de slag gegaan aan de overdracht, waarin ik mij bezig heb gehouden met het maken en schrijven van de readme met alle benodigdheden voor de opdrachtgever om uiteindelijk verder te gaan met het doorontwikkelen of andere die dat dan voor haar doen. Deze week heb ik weer veel bijgeleerd, dit keer meer omtrent het overdragen, want het moet wel duidelijk blijven voor de andere partij en ik vond het goed gaan.

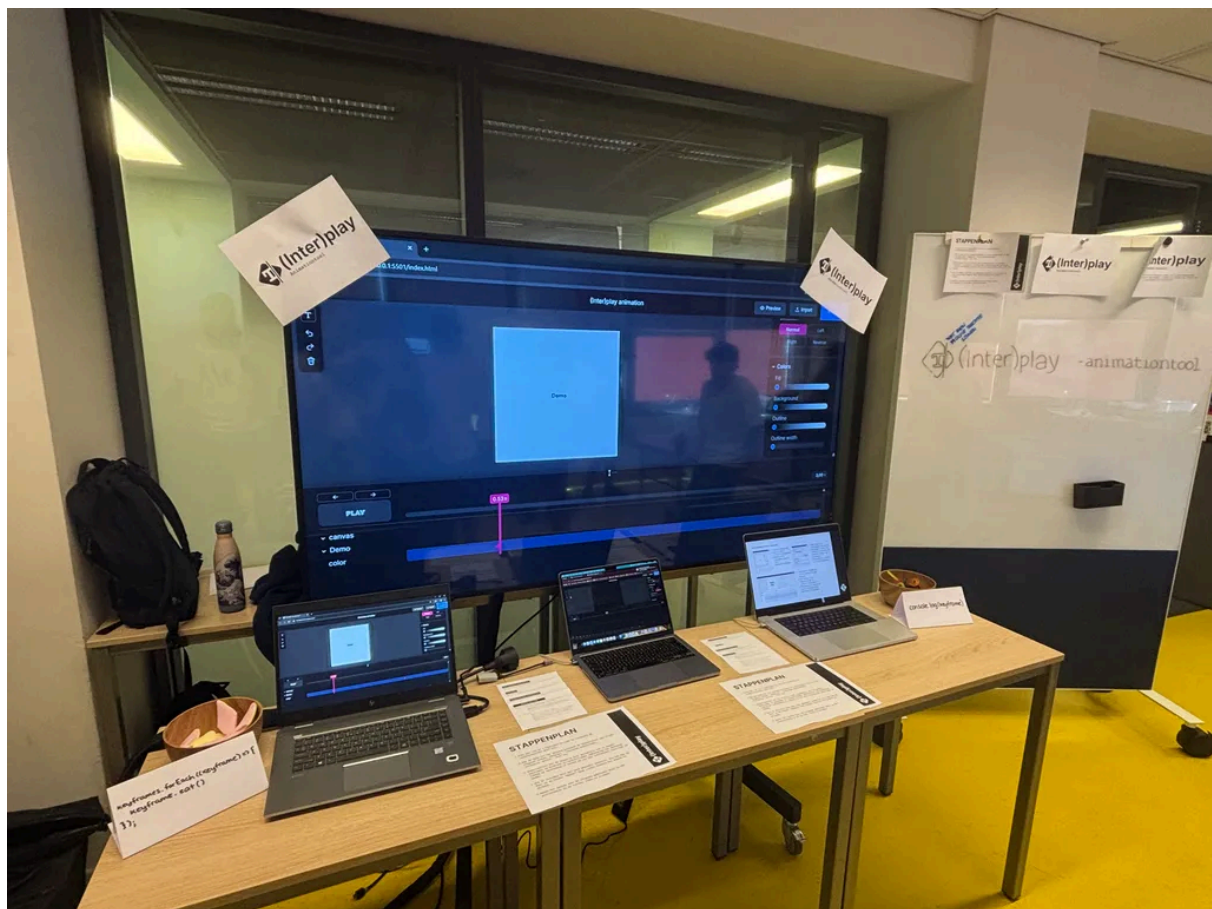
Eindoplevering met opdrachtgever

Op woensdag hadden we de meeting met de opdrachtgever, we hebben daarvoor een presentatie gemaakt, die mijn groepsgenoten hebben gemaakt terwijl ik de readme update en dit hebben we uitgebreid besproken met de opdrachtgever. We hebben bepaalde voor haar nieuwe keuzes toegelicht, het uiteindelijke product laten zien en eventuele recommendations gedaan wat handig zou kunnen zijn voor in de toekomst. Daarna had ze de kans om vragen

te stellen voor eventuele verduidelijking. En hebben we daarna alles wat nodig was naar haar gemaïld.

EXPO

Wij hebben uiteindelijk de opstelling voor de expo als volgt neergezet, met keyframe snoepjes om tussendoor lekker te snacken. Met een stappenplan hoe je een animatie het beste kan beginnen en kaartjes met begrippen waar je uiteindelijk mee kan oefenen als aansluiting op de PD van Marjolijn.



Reflectie meesterproef

Overall was dit een ontzettend uitdagende opdracht geweest, maar wel heel tof om te zien hoe wij tot dit eindproduct zijn gekomen. Ik heb ontzettend veel geleerd en aan mijn leerdoelen kunnen werken. Soms had ik zelf wel wat moeite om met de werkdruk om te gaan, maar uiteindelijk is dat wel verminderd door de backlog die wekelijks een beetje minder werd en wij elkaar goed aanvulde binnen het project.

Als voorbeeld genomen: de buildvizualiser is een enorme functie geworden. Dit had, als ik meer tijd had gehad, opgesplitst kunnen worden in meerdere functies. (Zie hieronder die functie) Dit is wel iets wat ik aan ga werken in het vervolg.

Wij kwamen namelijk als groep op een punt uit waarin wij sommige code van elkaar niet begrepen en niet uit konden leggen. Dus zijn wij gestart met het code reviewen zodat wij de code wel begrepen, dit hebben wij op deze manier gedaan, omdat het een enorme stap achteruit was in het project en wij via deze manier kleine suggesties in aanpassingen aan toe konden voegen.

Waarin ik wel een aantal keer duidelijk heb aangegeven dat er code aangepast moest worden, zoals if/elses en dat bepaalde benamingen anders neergezet moesten worden en meer comment gezet moesten worden om het voor mij duidelijk te maken.

Meest trots

Het meest trots ben ik op de manier waarop ik deze opdracht ben aangegaan, ik zag er namelijk best wel tegenop om een tijdlijn te maken en ik wist niet zo goed waar wij moesten beginnen. Maar uiteindelijk werkt het wel gewoon en dat geeft een fijn gevoel. Zeker ook omdat ik in het begin van dit project ook meteen begon aan die tijdlijn. Ik had niet verwacht voorafgaand aan deze opdracht om dit te maken.

Had beter gekund

Voor een volgende keer had ik naar mijn idee nog wel even wat meer betrokken mogen zijn bij de code van andere en zeker gedurende het proces van deze hele samenwerking, ook omdat ik de rol als codebeheerder op mij had genomen had dat het misschien een stuk makkelijker gemaakt.

Ook wil ik nog meer oefenen met het opsplitsen van een enorme functie in bruikbare nieuwe functies, dat is wel nog iets wat ik lastig vind. Als voorbeeld genomen de buildvizualiser is een enorme functie geworden, dit had als ik meer tijd had gehad opgesplitst kunnen worden in meerdere functies. (Zie hieronder die functie) Dit is wel iets wat ik aan ga werken in het vervolg.

```
function buildVisualizer() {  
  const container = document.querySelector(".timeline-container");
```

```

container.innerHTML = "";

// Size is the length of the map
if (animationData.getAnimation().animations.size === 0) r
eturn;

// Loopen through each element to create a track for ever
y single element and property
animationData.getAnimation().elements.forEach((value, ele
mentId) => {

    // Whitin the timeline create a dialog and put it in th
e container
    const timelineSection = document.createElement('detail
s');
    timelineSection.classList.add("timeline-section");
    timelineSection.open = true;

    // JavaScript function that lets the observer-object
("observer") observe/check if there is anything happening i
n the timelineSection
    observer.observe(timelineSection);

    // Add a summary which is the elementId or value and se
t it in the details
    const contentSummary = animationData.getElement(element
Id).content;
    const maxLengthSummary = 18;
    const timelineSectionSummary = document.createElement
('summary');

    // Gives the summary a max-length, when it is bigger, t
he rest of the characters will be replaced with three dots.
    if (contentSummary.length > maxLengthSummary) {
        timelineSectionSummary.innerHTML = contentSummary.sli
ce(0, maxLengthSummary) + "...";
    } else {
        timelineSectionSummary.innerHTML = contentSummary;
    }
}

```

```

    }

    timelineSection.appendChild(timelineSectionSummary);

    container.appendChild(timelineSection);

    // get the properties that are defined within the id
    const properties = animationData.getProperties(elementId);

    // For each property you make a new track and row within the timeline
    properties.forEach((propertyName) => {
        const keyframes = animationData.getKeyframes(elementId, propertyName);

        const row = document.createElement("div");
        row.classList.add("row");

        const track = document.createElement("div");
        track.classList.add("track");

        const label = document.createElement("p");
        label.classList.add("track-label");

        // Create text
        const labelText = document.createElement("span");
        labelText.textContent = propertyName;
        labelText.style.cursor = "pointer";

        // Create delete button
        const deleteBtn = document.createElement("span");
        deleteBtn.innerHTML = "&times;";
        deleteBtn.classList.add("delete-property-btn");
        deleteBtn.title = `Delete ${propertyName}`;
    });

```

```

// Toggle class that shows delete button
labelText.addEventListener("click", () => {
  deleteBtn.classList.toggle("show-delete");
});

// Call deleteProperty on click
deleteBtn.addEventListener("click", (event) => {
  // event.stopPropagation();
  animationData.deleteProperty(elementId, propertyName);

  createSnapshot(animationData);
});

// Add deletebutton and label text to the <p>
label.append(deleteBtn, labelText);

row.append(label, track);

// Create for each keyframe point a point on the row
keyframes.forEach((keyframe) => {
  const point = document.createElement("div");
  point.classList.add("keyframe", `key-${keyframe.id}`);

  point.style.setProperty("--p", keyframe.progress);

  track.appendChild(point);

  // Create a drag for the keyframe and update the value
  Draggable.create(point, {
    // Type of way you can drag the element on the x axis
    type: "x",
    bounds: track,
    onClick() {
      // Make the keyframe the active keyframe
      activeKeyId = keyframe.id;
    }
  });
});

```

```

        point.classList.add("active-keyframe");
        player.setProgress(keyframe.progress);
        buildVisualizer();
    },
    onDragEnd() {
        // Get the width from the track and the point element which stands for the keyframe
        const trackWidth = track.offsetWidth;
        const pointX = this.x + keyframe.progress * trackWidth;

        // Chatgpt to make the calculation: How to make the calculation for the new progress
        // calculate the Newprogress by defiding the currentpointX with the trackwidth, the value can't be above 1, and the value can't be below 0
        const newProgress = Math.max(0,
            Math.min(1, pointX / trackWidth)
        );

        // Moves keyframe within animation to a new place in time (newProgress).
        animationData.moveKeyframe(
            keyframe.id,
            newProgress
        );

        // Make the keyframe the active keyframe
        activeKeyId = keyframe.id;
        point.classList.add("active-keyframe");
        buildVisualizer();
    }
});

// Add the elements to the html
timelineSection.appendChild(row);
});

```

```
});  
});
```

Reflectie Leerdoelen

Leerdoel 1

Tijdens de meesterproef heb ik meermaals kunnen werken aan mijn opgestelde leerdoelen. De eerste leerdoel *"Tijdens het samenwerken mijn belangen beter benoemen en de andere daarin ook de ruimte in te geven. Zodat de communicatie in groepsverband beter voor iedereen prettig verloopt."* kwam in het begin veel aan bod waarin we de standaardafspraken maakten, maar ook gedurende het project.

Wij kwam namelijk als groep op een punt uit waarin wij sommige code van elkaar niet begrepen en niet uit konden leggen. Dus zijn wij gestart met het code reviewen zodat wij de code wel begrepen, dit hebben wij op deze manier gedaan, omdat het een enorme stap achteruit was in het project en wij via deze manier kleine suggesties in aanpassingen aan toe konden voegen. Waarin ik wel een aantal keer duidelijk heb eengegeven dat er code aangepast moest worden, zoals if/elses en dat bepaalde benamingen anders neergezet moesten worden en meer comment gezet moeten worden om het voor mij duidelijk te maken.

Ook hadden we als groep zijnde code conventions opgesteld, maar dit was naar mijn idee achteraf toch iets te weinig. Er had meer bijgemogen en meer op gelet mogen worden. Nu heb ik er zelf ook niet altijd opgelet en dat is voor een volgende keer wel belangrijk om meer te doen, zodat je als groep zijnde meer op één lijn zit.

Leerdoel 2

Vervolgens bij mijn tweede leerdoel had ik *"Aan het einde van de meesterproef wil ik duidelijkere code schrijven, zodat het overdraagbaar en makkelijk te begrijpen is voor een ander. Zeker met het schrijven van javascript code, ik begrijp nu wat ik zelf doe, maar anderen moeten het ook nog snappen."* Voor dit leerdoel had ik de taak als codebeheerder op mij genomen; ik zorgde voor de merges waarin conflicten plaatsvonden en die ging ik oplossen. Ik merkte uiteindelijk dat dit best complex kan worden dus ik heb er met regelmaat hulp bij gevraagd of iedereen of een iemand met mij mee wilde kijken hiervoor.

Ook heb ik in het begin even met Kerr meegekeken hoe hij het aanpakt om code overzichtelijk en duidelijk te maken, daarin gaf hij aan dat alle variabele het beste bovenin neer kon zetten en de functies die standaard bij het laden van de pagina daar net onder, zodat het een stuk overzichtelijker is.

De samenwerking verliep wel goed; we hadden even wat struggles op het gebied van dat de code mogelijk te ingewikkeld zou zijn. Maar uiteindelijk hebben we daar met z'n allen voor gezeten en snappen we het allemaal nu wel beter.

Als laatste heb ik de overdracht geschreven voor de opdrachtgever, ik heb de **readme** op mij genomen en op die manier een duidelijk en overzichtelijk document te maken, wat zij kan gebruiken bij het gebruiken van deze tool. Naar mijn idee ging dit wel makkelijk, het was daarnaast ook fijn voor mijzelf om dit allemaal te noteren.

Leerdoel 3

Daarnaast wil ik ook werken aan visuele esthetiek van een website, door mooie en interessante features eraan toe te voegen, die passend zijn bij het project. Een beetje het randje opzoeken van mijn kunnen en daarbij ook nog blijven werken aan mijn leerdoel van afgelopen weken het schrijven van redelijk complexe javascript/css code die ik ook begrijp.

Terugkijken op dit project denk ik dat ik grote stappen heb gezet met dit leerdoel. Door bewust te kiezen voor een uitdagend project, werkend met nieuwe methoden zoals classes en nieuwe library. Heb ik geleerd hoe ik iets complex op verschillende manieren kan aanpakken.

De visuele esthetiek heb ik wel wat minder aan kunnen werken, omdat ik mijn focus meer had gelegd op de complexe code, maar ik heb in bepaalde momenten wel mijn mening gedeeld over bepaalde keuzes binnen dit project. Zo was er een keuze gaande over dat de playhead deppink was, waardoor deze opviel ten opzichte van de tracks. Daar was ik het mee eens dat zo moest blijven en niet een andere kleur blauw om zo binnen de stijl te blijven.

In het begin van dit project ben ik gestart met het schrijven van de timeline code, zoals eerder benoemd. Dit was best lastig, omdat ik het en probeerde met een library waar ik nog nooit mee heb gewerkt en ik wist niet hoe ik aan zoiets moest beginnen. Uiteindelijk is het gelukt door goed in te lezen op de documentatie en ik heb gemerkt dat dat mij wel verder helpt in complexere dingen maken, daarnaast hebben we op een gegeven moment ook gehad dat

de code te complex werd en daardoor met zijn alle hebben gezeten om dit op te lossen, maar daardoor begrijp ik het wel een stuk beter.

Reflectie samenwerking

De samenwerking verliep wel goed, we hadden even wat struggles op het gebied van dat de code mogelijk te ingewikkeld zou zijn. Maar uiteindelijk hebben we daar met zijn alle voor gezeten en snappen we het allemaal nu wel beter. Verder waren we begonnen met het gebruiken van de backlog, dit was ontzettend fijn. Je kon gemakkelijk bijhouden wat er nog gedaan moet worden en wie waar mee bezig is, dat het een prettig overzicht geeft. Zeker in een project zo complex en best groot als deze was dat wel heel fijn.

Daarnaast stonden we allemaal wel open om elkaar te helpen als iemand er niet uitkwam en hebben we veelal van de merges gezamenlijk gedaan. Ondanks dat ik de codebeheerder was gaf dit toch wel een fijn gevoel, ook omdat ik soms niet wist wat we wel konden behouden en wat niet.

Overall heb ik ontzettend veel geleerd in de meesterproef. Ik heb wederom weer veel nieuwe dingen geleerd, net als de rest van de minor. Ik neem alle nieuwe kennis die ik heb opgedaan mee en ik ga nog verder werken aan de leerdoelen die ik hiervoor had gesteld, omdat dit niet leerdoelen zijn die je zou moeten herhalen en oefenen.

Tegen week twee aan begonnen we in de ochtend ook met het bekijken en opschonen van de backlog, eigenlijk een beetje een stand-up (maar dan zittend). Dit bood ook wederom weer een fijn en overzichtelijk gevoel. Iedereen was ook dagelijks aanwezig, omdat we vrijwel altijd op school aan het werk gingen en als er iets was dan zeiden de meeste dat van tevoren en anders spraken we er iemand op aan.

Ik ben ontzettend blij met hoe de samenwerking is verlopen in dit project.

Conclusie

Overall heb ik ontzettend veel geleerd in de meesterproef ik heb wederom weer veel nieuwe dingen geleerd, net als de rest van de minor. Ik neem alle nieuwe kennis die ik heb opgedaan mee en ik ga nog verder werken aan de leerdoelen die ik hiervoor had gesteld, omdat dit niet leerdoelen zijn die je zou moeten herhalen en oefenen. Ik ga nog verder werken aan mijn skill om op een juiste manier producten over te dragen, werken aan de visuele esthetiek van een website en het vooraf afspraken maken met teamgenoten.

